

CSS Flexbox Flex-Wrap & Flex-Flow

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What are flex-wrap and flex-flow?	2
1.2	Why Use flex-wrap and flex-flow?	2
1.3	Quick Comparison	2
2	flex-wrap Property	3
2.1	Understanding flex-wrap	3
2.1.1	Values	3
2.1.2	When flex-wrap Applies	3
2.2	nowrap (Default)	4
2.2.1	Syntax	4
2.2.2	Characteristics	4
2.2.3	Examples	4
2.3	wrap	5
2.3.1	Syntax	5
2.3.2	Characteristics	5
2.3.3	Examples	5
2.4	wrap-reverse	8
2.4.1	Syntax	8
2.4.2	Characteristics	8
2.4.3	Examples	8
3	flex-flow Shorthand	10
3.1	Understanding flex-flow	10
3.1.1	Syntax	10
3.1.2	Possible Combinations	10
3.2	Common flex-flow Values	10
3.2.1	row wrap (Most Common)	10
3.2.2	column wrap	11
3.2.3	row-reverse wrap	11
3.2.4	column-reverse wrap-reverse	11
3.3	flex-flow Examples	11
4	Practical Applications	13
4.1	Application 1: Responsive Product Grid	13
4.2	Application 2: Tag Input System	14
4.3	Application 3: Flexible Navigation	15

4.4	Application 4: Image Gallery	16
5	Responsive Behavior	18
5.1	Automatic Responsiveness	18
5.2	With Media Queries	18
5.3	Controlling Wrap Behavior	19
6	Advanced Techniques	20
6.1	Combining with align-content	20
6.2	Nested Wrapping	20
6.3	Dynamic Wrap with Content Queries	21
7	Troubleshooting	22
7.1	Issue 1: Items Not Wrapping	22
7.2	Issue 2: Uneven Last Row	22
7.3	Issue 3: Wrapped Lines Not Aligned	23
7.4	Issue 4: Content Overflow on Wrap	23
8	Best Practices	24
8.1	When to Use Each	24
8.2	Performance Considerations	24
8.3	Accessibility Tips	24
8.4	Code Organization	24
9	Summary: Key Takeaways	26
9.1	flex-wrap Values	26
9.2	flex-flow Syntax	26
9.3	When Items Wrap	26
9.4	Key Rules	27
9.5	Quick Reference	27
9.6	When to Use flex-wrap vs. Grid	28

Chapter 1

Introduction

1.1 What are flex-wrap and flex-flow?

flex-wrap controls whether flex items wrap to new lines when they run out of space. **flex-flow** is a shorthand property that combines **flex-direction** and **flex-wrap** into a single declaration.

Key Points:

- **flex-wrap** prevents item shrinking by forcing wrapping
- **flex-flow** saves code by combining two properties
- Essential for responsive design without media queries
- Enables automatic line breaks when needed

1.2 Why Use flex-wrap and flex-flow?

- Create responsive grids that adapt automatically
- Prevent content from overflowing or shrinking
- Build flexible multi-line layouts
- Simplify CSS with shorthand syntax
- Handle dynamic content gracefully
- Maintain item sizes while wrapping

1.3 Quick Comparison

flex-wrap	flex-flow
Single property	Shorthand (2 in 1)
Controls wrapping only	Controls direction + wrap
Combined with flex-direction	Combines both
More explicit	More concise

Chapter 2

flex-wrap Property

2.1 Understanding flex-wrap

flex-wrap defines how flex items are arranged when they can't fit on a single line.

2.1.1 Values

- nowrap - All items on one line (default)
- wrap - Items wrap to new lines
- wrap-reverse - Items wrap in reverse order

2.1.2 When flex-wrap Applies

```
/* Without flex-wrap (or nowrap) - items stay on one line */
.flex-container {
  display: flex;
  width: 400px;
}

.item {
  width: 150px; /* 3 items = 450px, exceeds 400px */
}

/* Items shrink to fit or overflow */

/* With flex-wrap: wrap - items move to new line */
.flex-container {
  display: flex;
  flex-wrap: wrap;
  width: 400px;
}

.item {
  width: 150px; /* Items stay 150px, wrap to next line */
}
```

2.2 nowrap (Default)

All items stay on a single line. Items may shrink or overflow.

2.2.1 Syntax

```
.flex-container {  
  display: flex;  
  flex-wrap: nowrap; /* Default behavior */  
}
```

2.2.2 Characteristics

- All items forced onto one line
- Items shrink if necessary to fit
- No wrapping occurs
- Items may overflow container
- Good for fixed layouts

2.2.3 Examples

Example 1: Navigation Bar (No Wrapping)

```
<nav class="navbar">  
  <div class="logo">Logo</div>  
  <a href="#">Home</a>  
  <a href="#">About</a>  
  <a href="#">Services</a>  
  <a href="#">Contact</a>  
</nav>  
  
.navbar {  
  display: flex;  
  flex-wrap: nowrap; /* Stay on one line */  
  gap: 20px;  
  background: #2c3e50;  
  padding: 15px;  
}  
  
.navbar a {  
  white-space: nowrap; /* Prevent text wrapping */  
}  
  
/* All nav items stay on one line, may shrink */
```

Example 2: Inline Toolbar

```
<div class="toolbar">  
  <button>Save</button>  
  <button>Edit</button>
```

```
<button>Delete</button>
<button>Export</button>
</div>

.toolbar {
  display: flex;
  flex-wrap: nowrap;
  gap: 10px;
}

.toolbar button {
  padding: 8px 16px;
  white-space: nowrap;
}

/* Buttons stay on one line */
```

2.3 wrap

Items wrap to new lines when necessary, maintaining their sizes.

2.3.1 Syntax

```
.flex-container {
  display: flex;
  flex-wrap: wrap;
}
```

2.3.2 Characteristics

- Items wrap to new lines when space is limited
- Items maintain their size (don't shrink)
- Creates multi-line layouts
- Perfect for responsive grids
- Most common value

2.3.3 Examples

Example 1: Product Grid

```
<div class="product-grid">
  <div class="product">
    
    <h3>Product 1</h3>
    <p>$29.99</p>
  </div>
  <div class="product">
    
```

```
        <h3>Product 2</h3>
        <p>$39.99</p>
    </div>
    <!-- More products -->
</div>

.product-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}

.product {
    flex: 1 1 200px; /* Minimum 200px, can grow */
    background: white;
    border-radius: 8px;
    overflow: hidden;
}

.product img {
    width: 100%;
    height: 150px;
    object-fit: cover;
}

/* Products arrange in rows, wrapping as needed */
```

Example 2: Badge/Tag Cloud

```
<div class="tag-cloud">
    <span class="tag">JavaScript</span>
    <span class="tag">React</span>
    <span class="tag">CSS</span>
    <span class="tag">Flexbox</span>
    <span class="tag">Web Design</span>
    <span class="tag">Responsive</span>
</div>

.tag-cloud {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
}

.tag {
    background: #3498db;
    color: white;
    padding: 6px 12px;
    border-radius: 20px;
    font-size: 12px;
    white-space: nowrap;
}

/* Tags wrap naturally when space is limited */
```


Example 3: Responsive Card Layout

```
<div class="card-container">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
  <div class="card">Card 4</div>
  <div class="card">Card 5</div>
</div>

.card-container {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}

.card {
  flex: 1 1 250px; /* Min 250px, grows to fill space */
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Automatically arranges to best fit available width */
```

Example 4: Flexible Button Group

```
<div class="button-group">
  <button>Previous</button>
  <button>Page 1</button>
  <button>Page 2</button>
  <button>Page 3</button>
  <button>Next</button>
</div>

.button-group {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.button-group button {
  padding: 8px 12px;
  background: #3498db;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  white-space: nowrap;
}

/* Buttons wrap to next line if needed */
```

2.4 wrap-reverse

Items wrap to new lines in reverse order.

2.4.1 Syntax

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```

2.4.2 Characteristics

- Items wrap similarly to `wrap`
- New lines appear above previous content
- Reverse visual order of wrapped lines
- Useful for chat-like interfaces
- Less common than `wrap`

2.4.3 Examples

Example 1: Reverse Wrapping Grid

```
<div class="reverse-grid">  
  <div class="item">Item 1</div>  
  <div class="item">Item 2</div>  
  <div class="item">Item 3</div>  
  <div class="item">Item 4</div>  
  <div class="item">Item 5</div>  
  <div class="item">Item 6</div>  
</div>  
  
.reverse-grid {  
  display: flex;  
  flex-wrap: wrap-reverse;  
  gap: 15px;  
}  
  
.item {  
  flex: 1 1 150px;  
  background: #3498db;  
  color: white;  
  padding: 20px;  
  border-radius: 8px;  
}  
  
/* Wrapped rows appear in reverse visual order */
```

Example 2: Chat-Like Interface

```
<div class="chat-container">
  <div class="message">First message</div>
  <div class="message">Second message</div>
  <div class="message">Third message</div>
</div>

.chat-container {
  display: flex;
  flex-wrap: wrap-reverse;
  flex-direction: column;
  gap: 10px;
  height: 300px;
}

.message {
  background: #3498db;
  color: white;
  padding: 10px 15px;
  border-radius: 8px;
  max-width: 70%;
}

/* Messages appear with latest at bottom */
```

Chapter 3

flex-flow Shorthand

3.1 Understanding flex-flow

flex-flow combines flex-direction and flex-wrap into a single property.

3.1.1 Syntax

```
/* flex-flow: flex-direction flex-wrap; */

.flex-container {
  display: flex;
  flex-flow: row wrap; /* direction wrap-value */
}

/* Equivalent to: */
.flex-container {
  flex-direction: row;
  flex-wrap: wrap;
}
```

3.1.2 Possible Combinations

Direction	Wrap	flex-flow
row	nowrap	row nowrap
row	wrap	row wrap
row	wrap-reverse	row wrap-reverse
column	nowrap	column nowrap
column	wrap	column wrap
column	wrap-reverse	column wrap-reverse

3.2 Common flex-flow Values

3.2.1 row wrap (Most Common)

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap; /* Horizontal, wraps downward */  
}  
  
/* Perfect for responsive grids */
```

3.2.2 column wrap

```
.flex-container {  
  display: flex;  
  flex-flow: column wrap; /* Vertical, wraps rightward */  
}  
  
/* Multi-column layout */
```

3.2.3 row-reverse wrap

```
.flex-container {  
  display: flex;  
  flex-flow: row-reverse wrap; /* RTL, wraps downward */  
}  
  
/* Reverse flow for RTL languages */
```

3.2.4 column-reverse wrap-reverse

```
.flex-container {  
  display: flex;  
  flex-flow: column-reverse wrap-reverse; /* Complex layout */  
}  
  
/* Bottom-to-top, right-to-left wrapping */
```

3.3 flex-flow Examples

Example 1: Simple Responsive Grid

```
<div class="grid">  
  <div class="card">Card 1</div>  
  <div class="card">Card 2</div>  
  <div class="card">Card 3</div>  
  <div class="card">Card 4</div>  
</div>  
  
.grid {  
  display: flex;  
  flex-flow: row wrap; /* One declaration instead of two */
```

```
        gap: 20px;
    }

    .card {
        flex: 1 1 250px;
        background: white;
        padding: 20px;
        border-radius: 8px;
    }

    /* Instead of:
        flex-direction: row;
        flex-wrap: wrap;
    */
```

Example 2: Multi-Column Layout

```
<div class="masonry">
    <div class="item">Item 1</div>
    <div class="item">Item 2</div>
    <div class="item">Item 3</div>
    <!-- More items -->
</div>

.masonry {
    display: flex;
    flex-flow: column wrap;
    max-height: 500px;
    gap: 15px;
}

.item {
    background: #3498db;
    padding: 20px;
    border-radius: 8px;
}

/* Items flow down, wrapping to next column */
```

Example 3: Bidirectional Content

```
/* For RTL languages, reverse flow with wrapping */
.rtl-content {
    display: flex;
    flex-flow: row-reverse wrap;
}

/* For LTR languages, normal wrapping */
.ltr-content {
    display: flex;
    flex-flow: row wrap;
}
```

Chapter 4

Practical Applications

4.1 Application 1: Responsive Product Grid

```
<div class="products">
  <div class="product">
    
    <h3>Product 1</h3>
    <p class="price">$29.99</p>
    <button>Add to Cart</button>
  </div>
  <!-- More products -->
</div>

.products {
  display: flex;
  flex-flow: row wrap;
  gap: 20px;
  padding: 20px;
}

.product {
  flex: 1 1 200px; /* Min 200px, can grow */
  background: white;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.product img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.product h3, .product p, .product button {
  margin: 0;
  padding: 10px 15px;
}
```

```
.product button {
  width: 100%;
  padding: 12px;
  background: #3498db;
  color: white;
  border: none;
  cursor: pointer;
}

/* Automatically scales from 1 to 5+ columns */
```

4.2 Application 2: Tag Input System

```
<div class="tag-input">
  <input type="text" placeholder="Add a tag...">
  <span class="tag">
    React
    <button class="tag-remove">&times;</button>
  </span>
  <span class="tag">
    JavaScript
    <button class="tag-remove">&times;</button>
  </span>
  <!-- More tags -->
</div>

.tag-input {
  display: flex;
  flex-flow: row wrap;
  gap: 8px;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
}

.tag-input input {
  flex: 1 1 auto;
  border: none;
  outline: none;
}

.tag {
  display: flex;
  align-items: center;
  gap: 6px;
  background: #3498db;
  color: white;
  padding: 4px 8px;
  border-radius: 12px;
  white-space: nowrap;
}
```



```
.tag-remove {
  background: none;
  border: none;
  color: white;
  cursor: pointer;
}

/* Tags wrap naturally as input grows */
```

4.3 Application 3: Flexible Navigation

```
<nav class="main-nav">
  <div class="logo">Logo</div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">Products</a>
    <a href="#">Services</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
  <div class="user-actions">
    <button>Login</button>
    <button>Sign Up</button>
  </div>
</nav>

.main-nav {
  display: flex;
  flex-flow: row wrap;
  align-items: center;
  gap: 20px;
  padding: 15px 30px;
  background: #2c3e50;
}

.logo {
  color: white;
  font-weight: bold;
  font-size: 20px;
}

.nav-links {
  display: flex;
  flex-flow: row wrap;
  gap: 15px;
  flex: 1;
}

.nav-links a {
  color: white;
  text-decoration: none;
  white-space: nowrap;
}
```

```
}

.user-actions {
  display: flex;
  flex-flow: row wrap;
  gap: 10px;
}

.user-actions button {
  padding: 8px 16px;
  background: #3498db;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

/* Wraps intelligently on narrow screens */
```

4.4 Application 4: Image Gallery

```
<div class="gallery">
  <figure class="gallery-item">
    
    <figcaption>Image 1</figcaption>
  </figure>
  <!-- More images -->
</div>

.gallery {
  display: flex;
  flex-flow: row wrap;
  gap: 15px;
  padding: 20px;
}

.gallery-item {
  flex: 1 1 200px;
  margin: 0;
}

.gallery-item img {
  width: 100%;
  height: 200px;
  object-fit: cover;
  border-radius: 8px;
}

.gallery-item figcaption {
  padding: 10px 0;
  text-align: center;
  font-size: 14px;
}
```

```
        color: #666;
    }

    /* Gallery adapts to any container width */
```

Chapter 5

Responsive Behavior

5.1 Automatic Responsiveness

With `flex-wrap: wrap`, layouts respond automatically to container width.

```
.responsive-grid {
  display: flex;
  flex-flow: row wrap;
  gap: 20px;
}

.item {
  flex: 1 1 250px; /* Key: flex-basis sets minimum */
}

/* 1200px wide: 4 items per row */
/* 800px wide: 3 items per row */
/* 500px wide: 2 items per row */
/* 300px wide: 1 item per row */

/* No media queries needed! */
```

5.2 With Media Queries

Combine `flex-flow` with media queries for finer control.

```
.grid {
  display: flex;
  flex-flow: row wrap;
  gap: 15px;
}

.item {
  flex: 1 1 150px;
}

/* Tablet: Increase basis */
@media (min-width: 768px) {
  .item {
```

```
        flex: 1 1 200px;
    }
}

/* Desktop: Larger items */
@media (min-width: 1200px) {
    .item {
        flex: 1 1 250px;
    }
}

/* Mobile: Smaller items */
@media (max-width: 480px) {
    .item {
        flex: 1 1 100%;
    }
}
```

5.3 Controlling Wrap Behavior

```
/* Wrap at medium sizes */
.flexible-wrap {
    display: flex;
    flex-flow: row wrap;
    gap: 20px;
}

.item {
    flex: 1 1 clamp(150px, 25%, 300px);
}

/* clamp ensures responsive sizing */
/* Min 150px, Preferred 25%, Max 300px */
```

Chapter 6

Advanced Techniques

6.1 Combining with align-content

`align-content` aligns wrapped lines along the cross axis.

```
.flex-container {
  display: flex;
  flex-flow: row wrap;
  height: 400px;
  align-content: center; /* Center wrapped lines vertically */
}

.item {
  flex: 1 1 100px;
}

/* Other align-content values:
- flex-start (default)
- flex-end
- center
- space-between
- space-around
- space-evenly
- stretch
*/
```

6.2 Nested Wrapping

```
<div class="outer">
  <div class="inner">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
  </div>
  <div class="inner">
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
  </div>
</div>
```

```
        </div>
    </div>

    .outer {
        display: flex;
        flex-flow: row wrap;
        gap: 20px;
    }

    .inner {
        display: flex;
        flex-flow: row wrap;
        gap: 10px;
        flex: 1 1 300px;
    }

    .item {
        flex: 1 1 50px;
        background: #3498db;
        color: white;
        padding: 10px;
        border-radius: 4px;
    }

    /* Multiple levels of wrapping */
```

6.3 Dynamic Wrap with Content Queries

```
.dynamic-grid {
    display: flex;
    flex-flow: row wrap;
    gap: 20px;
}

.item {
    flex: 1 1 auto;
    min-width: max-content;
}

/* Items only wrap when absolutely necessary */
```

Chapter 7

Troubleshooting

7.1 Issue 1: Items Not Wrapping

Problem: Items stay on one line and shrink instead of wrapping.

Solution: Use `flex-wrap: wrap` and set explicit `flex-basis`.

```
/* WRONG: Items shrink instead of wrap */
.container {
  display: flex;
  width: 400px;
}

.item {
  width: 150px; /* Shrinks to fit */
}

/* CORRECT: Items wrap */
.container {
  display: flex;
  flex-wrap: wrap;
  width: 400px;
}

.item {
  flex: 1 1 150px; /* Wraps at 150px */
}
```

7.2 Issue 2: Uneven Last Row

Problem: Last row of wrapped items stretches unevenly.

Solution: Use `justify-content: flex-start` or `gap` property.

```
/* GOOD: Items stay their size, justify left */
.container {
  display: flex;
  flex-wrap: wrap;
  justify-content: flex-start;
  gap: 20px;
}
```



```
.item {  
    flex: 0 1 200px; /* Fixed width */  
}  
  
/* ALTERNATIVE: Use gap instead of margins */  
.container {  
    display: flex;  
    flex-wrap: wrap;  
    gap: 20px;  
}
```

7.3 Issue 3: Wrapped Lines Not Aligned

Problem: Multiple wrapped lines have uneven spacing.

Solution: Use `align-content` property.

```
.container {  
    display: flex;  
    flex-wrap: wrap;  
    height: 500px;  
    align-content: flex-start; /* Align lines to top */  
    gap: 20px;  
}  
  
.item {  
    flex: 1 1 150px;  
}
```

7.4 Issue 4: Content Overflow on Wrap

Problem: Wrapped content overflows container.

Solution: Use `overflow: auto` or ensure container height.

```
.container {  
    display: flex;  
    flex-wrap: wrap;  
    max-height: 300px;  
    overflow-y: auto;  
}  
  
.item {  
    flex: 1 1 100px;  
}
```

Chapter 8

Best Practices

8.1 When to Use Each

- **nowrap:** Fixed layouts, navigation bars, toolbars
- **wrap:** Responsive grids, product listings, flexible content
- **wrap-reverse:** Chat interfaces, reverse content flow
- **flex-flow:** When combining direction and wrap changes

8.2 Performance Considerations

- Wrapping is efficient - use freely
- Prefer **flex-wrap: wrap** over fixed layouts
- Combine with **gap** for cleaner spacing
- Use **flex-basis** for wrap control

8.3 Accessibility Tips

- Maintain logical source order in HTML
- Test wrapping behavior at different sizes
- Ensure sufficient color contrast
- Wrap shouldn't hide important content
- Use semantic HTML within flex containers

8.4 Code Organization

```
/* GOOD: Clear flex declarations */
.flex-container {
  display: flex;
  flex-flow: row wrap;
  gap: 20px;
}

.flex-item {
  flex: 1 1 200px;
}

/* BAD: Implicit flex-flow */
.flex-container {
  display: flex;
  justify-content: space-between;
}

/* Unclear what happens on wrap */
```

Chapter 9

Summary: Key Takeaways

9.1 flex-wrap Values

Value	Behavior
nowrap	One line, items shrink/overflow
wrap	Multiple lines, items maintain size
wrap-reverse	Multiple lines, reverse order

9.2 flex-flow Syntax

```
/* flex-flow: [flex-direction] [flex-wrap] */  
  
.default { flex-flow: row nowrap; }  
.responsive { flex-flow: row wrap; }  
.vertical { flex-flow: column wrap; }  
.rtl { flex-flow: row-reverse wrap; }  
.reverse { flex-flow: column-reverse wrap-reverse; }
```

9.3 When Items Wrap

Items wrap when total width exceeds container, using flex-basis as minimum.

```
/* With flex-wrap: wrap */  
.container {  
  width: 500px;  
  flex-wrap: wrap;  
}  
  
.item {  
  flex: 1 1 150px; /* Minimum 150px */  
}  
  
/* 500px ÷ 150px = 3.33 items fit */
```

```
/* 4th item wraps to next line */
```

9.4 Key Rules

1. Default is `nowrap` - items don't wrap
2. Use `flex-wrap: wrap` for responsive layouts
3. Set `flex-basis` to control wrap point
4. `flex-flow` combines direction and wrap
5. Wrapping is performant - use freely
6. `align-content` aligns wrapped lines
7. `gap` provides clean spacing
8. Works great with responsive design
9. No media queries often needed
10. Combine with modern CSS for best results

9.5 Quick Reference

```
/* Responsive grid (most common) */
.grid {
  display: flex;
  flex-flow: row wrap;
  gap: 20px;
}

.item {
  flex: 1 1 250px; /* Wraps at 250px */
}

/* Vertical with wrapping */
.vertical {
  display: flex;
  flex-flow: column wrap;
  max-height: 500px;
}

/* No wrapping (fixed layout) */
.fixed {
  display: flex;
  flex-flow: row nowrap;
}

/* One-liner for most use cases */
.container {
```

```
display: flex;  
flex-flow: row wrap;  
gap: 15px;  
}
```

9.6 When to Use flex-wrap vs. Grid

- **flex-wrap:** One-dimensional, simple grids, responsive without media queries
- **CSS Grid:** Two-dimensional, complex layouts, precise control